VM16: A Minimal Portable 16-bit Virtual Machine

Specification, version 0.1 (draft)

October 27, 2026

Abstract

VM16 is a small register-and-stack virtual machine meant to be implemented in a few hundred lines of assembly language on 8- and 16-bit processors such as the Z80, 6502 and 8086. It is powerful enough to write a memory allocator, a garbage collector, hash tables, and an interpreter for a higher-level abstract machine such as an SECD machine or a combinator graph reducer. This document defines the machine state, the instruction encoding, the exact semantics of every instruction, the system-call interface and the program image format. A recommended calling convention and assembly syntax are given as non-normative appendices.

Contents

1 Goals and principles **2**

2 Machine state **2**

2.1 Registers . . . 3

2.2 Memory . . . 3

2.3 Stacks . . . 3

2.4 Initial state . . . 4

3 Instruction encoding **4**

4 Instruction semantics **4**

4.1 Opcode map . . . 4

4.2 Notes on individual instructions . . . 6

5 System calls **7**

6 Errors and undefined behaviour **7**

7 Program images **8**

8 Examples **8**

8.1 A cons-cell allocator . . . 8

8.2 Jump-table dispatch . . . 8

8.3 A complete program: 8! . . . 9

A Recommended calling convention (non-normative) **10**

B Assembly syntax (non-normative) 11

C Implementation notes (non-normative) 11

D Open questions for later versions 11

1 Goals and principles

P1. Small to implement. An interpreter should fit in 1–2 KB of native code on an 8-bit processor. Every instruction has a fixed, simple decoding.

P2. Identical on every host. The semantics never depend on host flags, word size or byte order. There is no flags register: conditional branches compare registers directly.

P3. Untyped. The machine knows words, bytes and addresses only. Heap allocation, garbage collection, tagging and data structures are written in VM16 code, as libraries.

P4. Precise roots. Return addresses live on a return stack separate from the data stack, so a garbage collector written in VM16 code can scan the data stack without mistaking a return address for a pointer.

P5. Good for writing interpreters. Indirect jumps and calls through registers support jump-table dispatch.

P6. An escape hatch. The `SYS` instruction gives access to host services and to native implementations of performance-critical routines.

Conventions. The key words *must*, *must not* and *may* are normative. Numbers written `0x1F` are hexadecimal. All arithmetic on addresses and register values is modulo 2^{16}.

2 Machine state

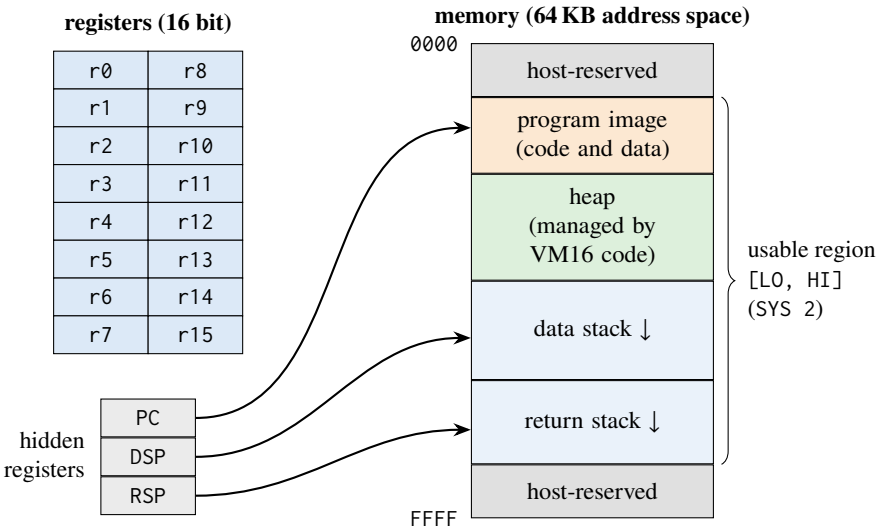


Figure 1: The VM16 machine state. The register file is not part of the address space. The layout of the usable region shown here is only an example: the program image header (section 7) chooses where the stacks start, and the program decides where its heap lives.

2.1 Registers

- Sixteen general-purpose 16-bit registers $r_0, \ldots, r_{15}$. No register has a special meaning to the hardware, except that `DIVU` writes its remainder to r_{15}.
- Three hidden 16-bit registers: the program counter PC, the data stack pointer DSP and the return stack pointer RSP. They are read and written only by the instructions that name them.
- The register file is *not* mapped into the address space. An implementation may keep it anywhere, for example in 6502 zero page.

2.2 Memory

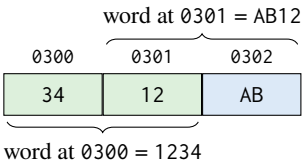
- Memory is an array of 2^{16} bytes. A VM16 address is used directly as a host address (identity mapping), so an implementation never has to translate addresses.
- Not all of it is available to the program: the host needs room for the interpreter, I/O and its own stack. The *usable region* is the contiguous range [LO, HI] reported by `SYS 2`. A program must not read or write outside it. Doing so is undefined behaviour (section 6).
- Words are 16 bits, little-endian, and need not be aligned:

$$\mathrm{mem}_{16}(a) = \mathrm{mem}_8(a) + 256 \cdot \mathrm{mem}_8(a + 1 \bmod 2^{16}).$$

  Storing a word writes the low byte at a and the high byte at $a + 1$.

  Example. Suppose the byte at `0x0302` is `0xAB`, and r_2 = `0x0300`.

```
LI r1, 0x1234
SW r1, [r2+0]   writes 0x34 at 0x0300 and 0x12 at 0x0301
LW r3, [r2+0]   r3 = 0x34 + 256 · 0x12 = 0x1234
LW r4, [r2+1]   unaligned: r4 = 0x12 + 256 · 0xAB = 0xAB12
LB r5, [r2+0]   r5 = 0x0034, the low byte only
```



  Addresses wrap around: the word at `0xFFFF` consists of the bytes at `0xFFFF` (low) and `0x0000` (high).

- Code and data share one memory. A store that modifies an instruction takes effect the next time that instruction is fetched. Programs that rely on this are not portable to implementations that translate VM16 code to native code (section C).

2.3 Stacks

Both stacks live in memory, grow towards lower addresses, and are *full descending*: the stack pointer holds the address of the item most recently pushed. Each item is one word.

$$\begin{aligned} &\text{push } v: && SP \leftarrow SP - 2;\ \ \mathrm{mem}_{16}(SP) \leftarrow v \\ &\text{pop:} && v \leftarrow \mathrm{mem}_{16}(SP);\ \ SP \leftarrow SP + 2 \end{aligned}$$

The machine does not check for overflow or underflow. It is up to the program image to give each stack enough room.

2.4 Initial state

When a program image is loaded (section 7), PC, DSP and RSP are taken from the image header, all of $r_0, \ldots, r_{15}$ are zero, and memory outside the image is unspecified.

3 Instruction encoding

Every instruction begins with a one-byte *opcode*, followed by zero to three operand bytes. There are eight formats (fig. 2).

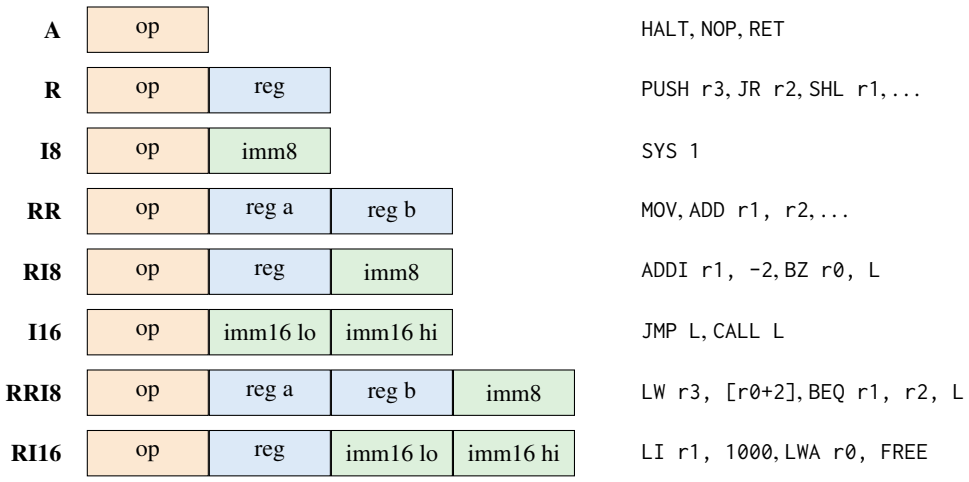


Figure 2: The eight instruction formats, one box per byte. Instructions are 1 to 4 bytes long and need not be aligned.

Register operands. A register operand byte holds *twice* the register number: r_n is encoded as $2n$, so the valid values are `0x00`, `0x02`, ..., `0x1E`. This lets an implementation use the byte directly as an offset into a register file of 2-byte entries, with no shifting or masking. Any other value is an invalid register operand (section 6).

Immediates. An *imm8* is one byte. Depending on the instruction it is used unsigned (0 to 255) or sign-extended (−128 to 127); section 4 says which. An *imm16* is a little-endian word.

Branch offsets. A branch's imm8 is a signed offset relative to the address of the *next* instruction. A branch can therefore reach −128 to +127 bytes from the end of the branch instruction. An assembler should replace an out-of-range conditional branch by the opposite condition branching over a `JMP`.

4 Instruction semantics

Notation. r_a and r_b are the registers named by the first and second register operands. *imm* is the immediate operand. sext(x) sign-extends an 8-bit value to 16 bits. *next* is the address of the following instruction. Every instruction first sets PC ← *next* and then performs the action shown. All arithmetic is modulo 2^{16}. In comparisons, *signed* means two's complement.

4.1 Opcode map

opcode	mnemonic	meaning	format	bytes	action
Control and moves					
`0x00`	`HALT`	halt	A	1	stop the machine; r_0 is the exit status
`0x01`	`NOP`	no operation	A	1	no operation
`0x02`	`LI ra, imm16`	load immediate	RI16	4	$r_a \leftarrow imm$
`0x03`	`MOV ra, rb`	move (copy a register)	RR	3	$r_a \leftarrow r_b$
Memory (offset *imm* is unsigned, 0 to 255)					
`0x04`	`LW ra, [rb+imm8]`	load word	RRI8	4	$r_a \leftarrow \mathrm{mem}_{16}(r_b + imm)$
`0x05`	`SW ra, [rb+imm8]`	store word	RRI8	4	$\mathrm{mem}_{16}(r_b + imm) \leftarrow r_a$
`0x06`	`LB ra, [rb+imm8]`	load byte	RRI8	4	$r_a \leftarrow \mathrm{mem}_{8}(r_b + imm)$, zero-extended
`0x07`	`SB ra, [rb+imm8]`	store byte	RRI8	4	$\mathrm{mem}_{8}(r_b + imm) \leftarrow r_a \bmod 256$
`0x08`	`LWA ra, imm16`	load word from absolute address	RI16	4	$r_a \leftarrow \mathrm{mem}_{16}(imm)$
`0x09`	`SWA ra, imm16`	store word to absolute address	RI16	4	$\mathrm{mem}_{16}(imm) \leftarrow r_a$
Arithmetic and logic					
`0x10`	`ADD ra, rb`	add	RR	3	$r_a \leftarrow r_a + r_b$
`0x11`	`SUB ra, rb`	subtract	RR	3	$r_a \leftarrow r_a - r_b$
`0x12`	`AND ra, rb`	bitwise and	RR	3	$r_a \leftarrow r_a \,\&\, r_b$
`0x13`	`OR ra, rb`	bitwise or	RR	3	$r_a \leftarrow r_a \mid r_b$
`0x14`	`XOR ra, rb`	bitwise exclusive or	RR	3	$r_a \leftarrow r_a \oplus r_b$
`0x15`	`MUL ra, rb`	multiply	RR	3	$r_a \leftarrow (r_a \times r_b) \bmod 2^{16}$
`0x16`	`DIVU ra, rb`	divide, unsigned	RR	3	unsigned division, see section 4.2
`0x18`	`NOT ra`	bitwise not	R	2	$r_a \leftarrow r_a \oplus$ `0xFFFF`
`0x19`	`NEG ra`	negate	R	2	$r_a \leftarrow 0 - r_a$
`0x1A`	`SHL ra`	shift left	R	2	$r_a \leftarrow 2r_a \bmod 2^{16}$
`0x1B`	`SHR ra`	shift right, logical	R	2	$r_a \leftarrow \lfloor r_a/2 \rfloor$ (logical: bit 15 becomes 0)
`0x1C`	`SAR ra`	shift right, arithmetic	R	2	arithmetic shift right by one (bit 15 is kept)
`0x1D`	`ADDI ra, imm8`	add immediate	RI8	3	$r_a \leftarrow r_a + \mathrm{sext}(imm)$
Branches and jumps (branch offsets are signed, relative to *next*)					
`0x20`	`BEQ ra, rb, rel8`	branch if equal	RRI8	4	if $r_a = r_b$ then PC $\leftarrow$ *next* + sext(*imm*)
`0x21`	`BNE ra, rb, rel8`	branch if not equal	RRI8	4	if $r_a \neq r_b$ then branch
`0x22`	`BLT ra, rb, rel8`	branch if less than (signed)	RRI8	4	if $r_a < r_b$ (signed) then branch
`0x23`	`BGE ra, rb, rel8`	branch if greater or equal (signed)	RRI8	4	if $r_a \geq r_b$ (signed) then branch
`0x24`	`BLTU ra, rb, rel8`	branch if less than, unsigned	RRI8	4	if $r_a < r_b$ (unsigned) then branch
`0x25`	`BGEU ra, rb, rel8`	branch if greater or equal, unsigned	RRI8	4	if $r_a \geq r_b$ (unsigned) then branch
`0x26`	`BZ ra, rel8`	branch if zero	RI8	3	if $r_a = 0$ then branch
`0x27`	`BNZ ra, rel8`	branch if not zero	RI8	3	if $r_a \neq 0$ then branch
`0x28`	`JMP imm16`	jump	I16	3	PC $\leftarrow$ *imm*
`0x29`	`JR ra`	jump to address in register	R	2	PC $\leftarrow r_a$
Stacks					
`0x30`	`PUSH ra`	push onto data stack	R	2	DSP $\leftarrow$ DSP – 2; $\mathrm{mem}_{16}(\mathrm{DSP}) \leftarrow r_a$

opcode	mnemonic	meaning	format	bytes	action
`0x31`	`POP ra`	pop from data stack	R	2	$r_a \leftarrow \text{mem}_{16}(\text{DSP})$; DSP $\leftarrow$ DSP + 2
`0x32`	`GETDSP ra`	get data stack pointer	R	2	$r_a \leftarrow$ DSP
`0x33`	`SETDSP ra`	set data stack pointer	R	2	DSP $\leftarrow r_a$
`0x34`	`GETRSP ra`	get return stack pointer	R	2	$r_a \leftarrow$ RSP
`0x35`	`SETRSP ra`	set return stack pointer	R	2	RSP $\leftarrow r_a$
Subroutines and system					
`0x38`	`CALL imm16`	call subroutine	I16	3	RSP $\leftarrow$ RSP − 2; $\text{mem}_{16}(\text{RSP}) \leftarrow$ *next*; PC $\leftarrow$ *imm*
`0x39`	`CALLR ra`	call subroutine at address in register	R	2	$t \leftarrow r_a$; push *next* on the return stack; PC $\leftarrow t$
`0x3A`	`RET`	return from subroutine	A	1	PC $\leftarrow \text{mem}_{16}(\text{RSP})$; RSP $\leftarrow$ RSP + 2
`0x3F`	`SYS imm8`	system call	I8	2	host service number *imm* (unsigned), see section 5

All opcodes not listed are *reserved*. Executing a reserved opcode is an illegal instruction (section 6).

4.2 Notes on individual instructions

Operand order. In all two-register instructions the first register is the destination, or the data register for loads and stores, and the second is the source or the base address. The order in the encoding is the order in the assembly syntax.

`POP`. `POP` always reads the stack before changing DSP.

`DIVU`. Let $n = r_a$ and $d = r_b$, both read before anything is written.

- If $d \neq 0$: $q = \lfloor n/d \rfloor$ and $m = n \bmod d$.
- If $d = 0$: $q =$ `0xFFFF` and $m = n$. Division by zero is not an error.

Then $r_{15} \leftarrow m$, and *afterwards* $r_a \leftarrow q$. If r_a is r_{15}, the quotient is the value that remains.

`MUL`. Only the low 16 bits of the product are kept. The result is the same whether the operands are viewed as signed or unsigned.

`CALLR` and `JR`. The target is read from the register before anything else changes. The target must be the address of an instruction.

`HALT`. The value of r_0 is reported to the host as the exit status, and 0 means success. How the host shows it is implementation-defined.

5 System calls

`SYS n` requests host service number n. Arguments are passed in r_0–r_3, and results are returned in r_0 and r_1. A system call must not change any register other than its stated results, nor DSP, RSP or memory, except as stated.

n	name	effect
0	`PUTC`	write the byte r_0 mod 256 to the output stream
1	`GETC`	$r_0 \leftarrow$ next byte of the input stream (0–255), or `0xFFFF` at end of input
2	`MEMINFO`	$r_0 \leftarrow$ LO, $r_1 \leftarrow$ HI, the first and last usable addresses
3	`DEBUG`	implementation-defined diagnostic, for example printing the registers. Must otherwise behave as `NOP`
4–31		reserved for future standard services
32–127		implementation-defined, for example native versions of library routines
128–255		reserved for programs: an implementation must treat these as illegal instructions unless configured otherwise

Requesting an unassigned service number is an illegal instruction.

Native routines. Services 32–127 exist so that an implementation can replace a hot routine written in VM16 code, such as a cell allocator or the mark phase of a collector, with native code. A program that uses them should keep the VM16 version and use the native one only when it is present. How a program detects that a service is present is left to a later version of this specification.

6 Errors and undefined behaviour

Illegal instructions. A *conforming* implementation must stop the machine and report an error when it fetches a reserved opcode, or when `SYS` names an unassigned service. With a jump table over all 256 opcodes this check costs nothing. The report should include PC and the offending byte.

Undefined behaviour. The following are undefined, and a conforming implementation may do anything:

- an invalid register operand byte (odd, or greater than `0x1E`);
- reading, writing, or executing outside the usable region;
- stack overflow or underflow beyond the space the program allows;
- `JR`, `CALLR` or `RET` to an address that is not the start of an instruction.

Checking implementations. An implementation may also detect some undefined behaviour and stop with an error. This is recommended for the reference implementation and for debugging builds, using these codes:

code	meaning
1	reserved opcode
2	invalid register operand
3	unassigned `SYS` service
4	memory access outside the usable region
5	data stack out of bounds
6	return stack out of bounds

7 Program images

A program image is a 14-byte header followed by the image body. All header fields are little-endian words.

offset	field	meaning
0	`magic`	the ASCII bytes `V16` followed by the byte `0x00` (format version 0)
4	`load`	address at which the body is loaded
6	`length`	number of bytes in the body
8	`entry`	initial PC
10	`dsp`	initial DSP
12	`rsp`	initial RSP

The loader must refuse an image whose body does not fit in the usable region. Because stacks are full descending, the initial stack pointers point one word *above* the first item to be pushed, which may be just past the end of the region the program has set aside for that stack.

8 Examples

8.1 A cons-cell allocator

Cells are two words, and free cells are linked through their second word. The word at `FREE` (here `0x0200`) points to the first free cell, or is 0 when there are none. Assembled at `0x0300`, with the collector `gc` at `0x0400`:

address	bytes	label	instruction	comment
`0300`	`08 00 00 02`	`cons:`	`LWA r0, FREE`	head of free list
`0304`	`26 00 11`		`BZ r0, cons_gc`	none left: collect
`0307`	`04 06 00 02`		`LW r3, [r0+2]`	next free cell
`030B`	`09 06 00 02`		`SWA r3, FREE`	unlink the cell
`030F`	`05 02 00 00`		`SW r1, [r0+0]`	car
`0313`	`05 04 00 02`		`SW r2, [r0+2]`	cdr
`0317`	`3A`		`RET`	
`0318`	`38 00 04`	`cons_gc:`	`CALL gc`	refills the free list
`031B`	`28 00 03`		`JMP cons`	and try again

The branch offset `0x11` is `0x0318` − `0x0307`: the target minus the address of the instruction after the branch. A collector that can fail to free anything must report that, rather than loop forever. Here that is left to `gc`.

8.2 Jump-table dispatch

The inner loop of an interpreter written in VM16 code, such as a combinator reducer dispatching on a node tag in r_1:

```
SHL r1          tag ×2 = offset into a table of words
LI r2, table
ADD r2, r1
LW r2, [r2+0]   handler address
JR r2           or CALLR r2 to return here afterwards
```

8.3 A complete program: 8!

The program below, `fact8.asm`, computes 8! = 40320 and prints it in decimal. It uses most of the machine's features:

- `fact` is recursive. It saves its argument on the data stack with `PUSH` and `POP`, while `CALL` and `RET` use the return stack.
- `print_u16` converts a number to decimal with `DIVU`, which leaves the quotient in the destination register and the remainder in r_{15}. The digits come out from the right, so they are pushed and then popped in printing order.
- `print_str` prints a zero-terminated string with `LB` and `SYS 0`.

It follows the calling convention of appendix A, and is written in the assembly syntax of appendix B. 8! is the largest factorial that fits in 16 bits.

```
; fact8.asm - compute 8! and print it in decimal.
;
;   python vm16asm.py fact8.asm      ->  fact8.v16
;   ./vm16 fact8.v16                 ->  8! = 40320
;
; 8! = 40320 still fits in 16 bits (unsigned). 9! would not.
; Registers follow the recommended calling convention: arguments and
; results in r0, r0-r7 and r15 may be changed by a call.

        .org    0x2000
        .entry  start
        .stacks 0xE000, 0xF000          ; data stack below E000, return stack below F000

start:  LI      r0, msg
        CALL    print_str               ; "8! = "
        LI      r0, 8
        CALL    fact                    ; r0 = 8!
        CALL    print_u16               ; print it
        LI      r0, '\n'
        SYS     0
        LI      r0, 0                   ; exit status 0
        HALT

; fact: r0 = r0!  Recursive: n! = n * (n-1)!, with 0! = 1! = 1.
fact:   LI      r1, 2
        BLTU    r0, r1, fact_one        ; n < 2
        PUSH    r0                      ; save n on the data stack
        ADDI    r0, -1
        CALL    fact                    ; r0 = (n-1)!
        POP     r1                      ; r1 = n
        MUL     r0, r1
        RET
fact_one:
        LI      r0, 1
        RET

; print_u16: print r0 as an unsigned decimal number.
; Divides by 10 repeatedly; the remainders are the digits from the right,
; so they are pushed and then popped in reverse order.
print_u16:
        LI      r1, 10
        LI      r2, 0                   ; number of digits
pu_div: DIVU    r0, r1                  ; r0 = r0 / 10, r15 = r0 mod 10
        PUSH    r15
        ADDI    r2, 1
        BNZ     r0, pu_div
pu_out: POP     r0
        ADDI    r0, '0'
```

```
        SYS     0
        ADDI    r2, -1
        BNZ     r2, pu_out
        RET

; print_str: print the zero-terminated string at address r0.
print_str:
        MOV     r1, r0
ps_loop:
        LB      r0, [r1]
        BZ      r0, ps_done
        SYS     0
        ADDI    r1, 1
        JMP     ps_loop
ps_done:
        RET

msg:    .ascii  "8! = "
        .byte   0
```

Assembling and running it (non-normative). The repository that holds this specification also contains an assembler, `vm16asm.py`, and a portable interpreter in C, `vm16.c`. From that directory:

```
cc -O2 -Wall -Wextra -o vm16 vm16.c    # build the interpreter, once
python3 vm16asm.py fact8.asm           # assemble: writes the image fact8.v16
./vm16 fact8.v16                       # run it; prints  8! = 40320
```

The image is 113 bytes plus the 14-byte header, and the run takes 146 instructions. Further options:

```
python3 vm16asm.py fact8.asm -l fact8.lst  # also write a listing: address, bytes, source
./vm16 -s fact8.v16                        # report instructions executed and speed
./vm16 -c fact8.v16                        # checking implementation (section 6)
./vm16 -r 2000-EFFF fact8.v16              # restrict the usable region, LO-HI in hex
echo $?                                    # exit status: r0 mod 256 after HALT
```

After a trap, `vm16` prints the trap code and PC and exits with status 128 + code. It exits with status 125 if it rejects the image.

A Recommended calling convention (non-normative)

The machine does not enforce any convention. Library code is easier to combine if it follows this one:

- Arguments in r_0–r_3, further arguments on the data stack. The result is returned in r_0, with a second result, if any, in r_1.
- r_0–r_7 and r_{15} may be changed by a call (caller-saved). r_8–r_{14} must be preserved (callee-saved), typically by `PUSH`/`POP`.
- DSP must be the same after a call as before, apart from arguments the callee is documented to consume.
- A garbage collector may treat every word between DSP and the base of the data stack as a possible root. Code that stores non-pointer data on the data stack must make it distinguishable from pointers, for example with the tagging scheme the collector uses.

B Assembly syntax (non-normative)

- One instruction per line: `label: MNEMONIC operands ; comment`. Mnemonics and register names are case-insensitive.
- Memory operands are written `[rb+n]`, and `[rb]` means `[rb+0]`. Branch and jump targets are labels or expressions.
- Directives: `.org` *addr*, `.equ` *name, value*, `.byte`, `.word`, `.ascii ". . . "`, `.space` *n*, `.entry` *label*, `.stacks` *dsp, rsp*. The last two fill in the image header. `.include "file"` assembles the lines of *file* in its place; the path is relative to the including file.
- Numbers are decimal, `0x`-prefixed hexadecimal, or character literals such as `'A'`.

C Implementation notes (non-normative)

6502
Keep the 32-byte register file in zero page, together with pointers for PC, DSP and RSP. A register operand byte can then be used directly as a zero-page index: `lda regs,x`. The 6502's own stack is only 256 bytes, which is why both VM16 stacks live in main memory.

Z80
Point `IX` at the register file and use `ld l,(ix+n)` only for fixed offsets. For operand-selected registers, form the address in `HL` from the operand byte. The Z80's hardware stack can stay private to the interpreter.

8086
Run in one 64 KB segment, with `DS` = `ES` = the VM16 memory, so VM16 addresses are offsets. The register file can be an array addressed by `BX`.

Dispatch
Use a 256-entry table of handler addresses indexed by the opcode byte, with all reserved entries pointing to the illegal-instruction handler.

Translation
Because every instruction has a fixed meaning that does not depend on interpreter state, VM16 code can be translated to native code by expanding each instruction into a fixed sequence. The exceptions are self-modifying code and `JR`/`CALLR`/`RET`, whose targets are VM16 addresses that must be mapped to native addresses at run time.

D Open questions for later versions

- Whether to add add-with-carry, for multi-word arithmetic such as bignums.
- Whether signed load/store offsets or a register-plus-register addressing mode would pay for themselves.
- How a program discovers which implementation-defined `SYS` services exist.
- Whether 8 registers would be enough, which would allow a denser encoding.
- Interrupts or timers, which version 0.1 deliberately leaves out.